

Rapport individuel

Mini-Projet GANs et Diffusion

Contribution : Dataset Fashion-MNIST et generation avec un GAN

Dylan Peltier

M2 IA & Big Data — ESIEE-IT Pontoise

2025–2026

1 Perimetre de la contribution

Le mini-projet est decoupe en quatre parties techniques (dataset, GAN, diffusion, comparaison/erreurs) et une partie d’analyse critique. Ma contribution couvre les deux premieres parties : la mise en place du **dataset Fashion-MNIST** et l’implementation complete d’un **GAN** pour la generation d’images de cette base. Les artefacts que je transmets aux autres membres du groupe sont :

- un pipeline de chargement et de pretraitement reproductible (`seed=42`, normalisation $[-1, 1]$);
- les poids du generateur et du discriminateur (`gan_generator.pth`, `gan_discriminator.pth`);
- un lot de 100 images generees (`gan_images_for_analysis.pth`) qui sert d’entree a la partie comparaison et a l’etude des erreurs.

2 Section Dataset — Fashion-MNIST

2.1 Choix et caracteristiques

Fashion-MNIST est un dataset de 70 000 images en niveaux de gris, de taille 28×28 pixels, reparties en 10 classes vestimentaires : T-shirt, Pantalon, Pull, Robe, Manteau, Sandale, Chemise, Sneaker, Sac et Bottine. Le split standard fournit 60 000 images d’entrainement et 10 000 images de test. C’est le dataset preconise par la fiche de projet : il est leger (entrainement realiste sur GPU grand public), accessible directement via `torchvision.datasets.FashionMNIST`, et structurellement plus riche que MNIST chiffres, ce qui permet d’observer de vraies pathologies generatives (mode collapse, hybridation de classes).

2.2 Pipeline implemente

J’ai concu un pipeline minimal et reproductible :

Listing 1 – Pretraitement Fashion-MNIST

```
1 transform = transforms.Compose([
2     transforms.ToTensor(),
3     transforms.Normalize((0.5,), (0.5,)), # [-1, 1] pour matcher Tanh
4 ])
```

Le choix d’une normalisation $[-1, 1]$ n’est pas neutre : il est dicte par la sortie **Tanh** du generateur (section suivante). Une normalisation $[0, 1]$ aurait force a utiliser une **Sigmoid** en sortie du generateur, qui est notoirement moins stable a l’entrainement d’un GAN. Cette coherence entre la normalisation du dataset et l’activation de sortie du generateur est une condition tacite mais essentielle.

2.3 Verification quantitative

Le batch échantillonné respecte la spécification : valeurs dans $[-1.0, 1.0]$ exactement, moyenne ≈ -0.43 (logique : la majorité des pixels sont en noir, valeur -1), écart-type ≈ 0.70 . La figure 1 montre la distribution des classes et un échantillon. Le dataset est *parfaitement équilibré* (6 000 images de train et 1 000 de test par classe), donc aucun reweighting n'est nécessaire.

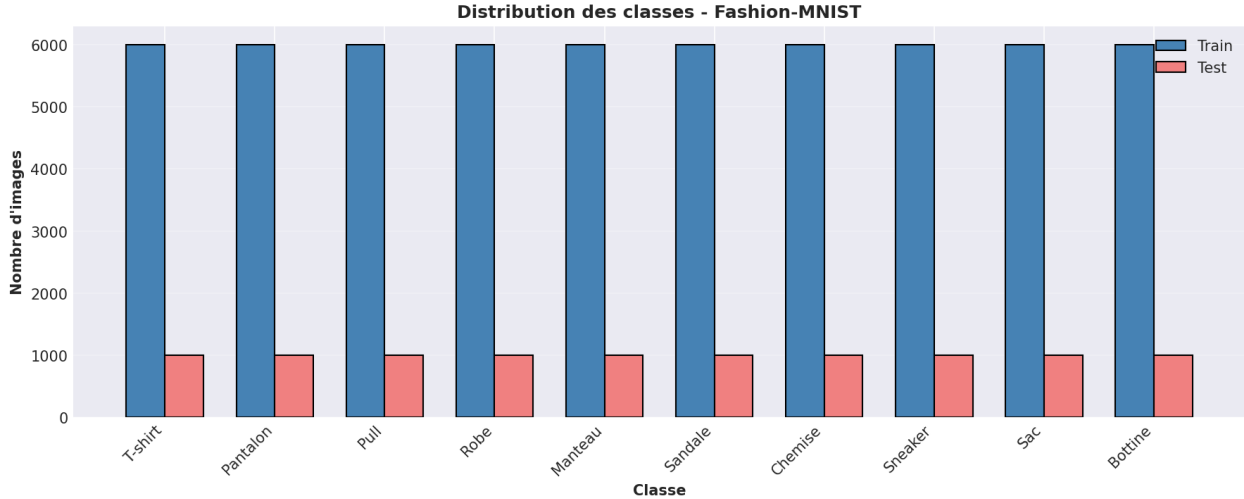


FIGURE 1 – Distribution des classes Fashion-MNIST : 6 000 (train) et 1 000 (test) par classe.

2.4 Analyse des difficultés anticipées

Classes difficiles à générer. La similarité inter-classes est le principal danger : T-shirt, Pull, Chemise et Manteau partagent la même silhouette de base (corps + manches), ce qui pousse un modèle non conditionné à produire des images intermédiaires non identifiables. À l'inverse, les classes structurellement séparées dans l'espace des images (Pantalon, Sandale, Sac) sont plus simples à générer.

Détails visuels délicats. À 28×28 en niveaux de gris, les détails suivants sont sous-représentés en signal et faciles à dégrader : lacets de chaussures, sangles de sacs, boutons, cols, mailles tricotées, motifs imprimés. Le modèle tend à les remplacer par des zones uniformes.

Difficultés intrinsèques pour un modèle génératif. La faible résolution écrase l'information fine sur très peu de pixels, ce qui rend la reconstruction de détails ambiguë. L'absence de couleur supprime un signal discriminant entre classes, et la multi-modalité intra-classe (chaussure de côté, de face, avec talon ou plate) favorise le mode collapse.

3 Section Partie 1 — Génération avec un GAN

3.1 Choix d'architecture : DCGAN

J'ai retenu une architecture **DCGAN** (Radford et al., 2015) adaptée au format 28×28 plutôt qu'un GAN purement dense. Le DCGAN est devenu la baseline canonique pour les images naturelles parce qu'il combine plusieurs choix qui font converger l'entraînement de manière fiable : convolutions transposées avec strides au lieu d'upsampling explicite, **BatchNorm** entre couches, **ReLU** (générateur) / **LeakyReLU** (discriminateur), pas de couche *fully-connected* à part la projection initiale, et **Tanh** en sortie.

Generateur. Projection dense $z \in \mathbb{R}^{100}$ vers un tenseur $7 \times 7 \times 256$, puis deux `ConvTranspose2d` (stride 2) qui doublent la resolution a chaque etape, $7 \rightarrow 14 \rightarrow 28$. Chaque couche intermediaire est suivie de `BatchNorm2d` + `ReLU`. La sortie est lisee par une `Conv2d` 3×3 suivie d'une `Tanh`. Total : 4,88 M parametres.

Discriminateur. Trois `Conv2d` avec strides ($28 \rightarrow 14 \rightarrow 7 \rightarrow 4$), `LeakyReLU(0.2)` et `BatchNorm2d` entre couches, suivies d'une projection dense vers un logit scalaire. J'utilise `BCEWithLogitsLoss` plutot que `Sigmoid` + `BCELoss` pour la stabilite numerique (la version `Logits` est numeriquement plus saine quand les logits sont extremes). Total : 1,07 M parametres.

Vecteur latent. J'ai fixe `latent_dim = 100`, valeur standard du papier DCGAN. Une dimension inferieure (16–32) tend a limiter la diversite generee ; une dimension superieure (256+) augmente la difficulte d'optimisation sans gain mesurable sur un dataset aussi simple.

3.2 Hyperparametres et boucle d'entrainement

Parametre	Valeur
Dimension latente	100
Batch size	128
Nombre d'epochs	50
Optimiseur	Adam ($\beta_1 = 0,5$, $\beta_2 = 0,999$)
Learning rate G et D	2×10^{-4}
Loss	<code>BCEWithLogitsLoss</code> (objectif non saturant)
Initialisation	$\mathcal{N}(0, 0,02)$ pour Conv et Linear
Seed	42 (Python, NumPy, PyTorch, CUDA)

TABLE 1 – Hyperparametres retenus pour l'entrainement du DCGAN.

A chaque iteration je realise (i) une mise a jour du discriminateur sur `BCE(D(reel), 1) + BCE(D(faux), 0)`, puis (ii) une mise a jour du generateur sur `BCE(D(faux), 1)` (objectif non saturant de Goodfellow). Pour suivre l'evolution du generateur de maniere comparable d'une epoch a l'autre, je stocke un snapshot a partir d'un **bruit fixe** `fixed_noise` en fin de chaque epoch ; cela isole l'effet de l'apprentissage de l'effet de l'aleatoire du sampling.

3.3 Resultats et lecture des courbes

La figure 2 montre un patron classique : le discriminateur *prend l'avantage* progressivement. La `D_loss` converge vers une zone stable autour de 0,5–1,0 (rappel : l'equilibre theorique ideal est $2 \ln 2 \approx 1,39$), tandis que la `G_loss` derive a la hausse, de ≈ 2 en debut a ≈ 3 –4 a la fin, avec des pics ponctuels jusqu'a 8. Ces pics ne sont pas des divergences durables : la boucle se restabilise immediatement apres, ce qui indique que le generateur reste capable de tromper le discriminateur la majorite du temps, malgre des episodes courts ou il echoue.

Lecon pratique sur les losses GAN. Contrairement a un classifieur, les losses GAN ne sont pas un proxy de la qualite. Une `D_loss` qui chuterait vers 0 serait une mauvaise nouvelle (le discriminateur ecrase le generateur, le gradient du generateur s'eteint) ; une `G_loss` qui chuterait brutalement traduirait souvent un mode collapse. Ici, le scenario observe (`D_loss` stable, `G_loss` en derive douce) est interpretable comme un equilibre encore exploitable mais qui se degrade en faveur du discriminateur — un entrainement plus long demanderait soit un ratio $D:G$ inferieur (mise a jour de G deux fois par iteration), soit du label smoothing pour ralentir D .

La figure 3 confirme la convergence visuelle : a l'epoch 1, les sorties sont des blobs flous mais possedent deja les axes spatiaux corrects (silhouettes globales esquissees). Des l'epoch 7, sept des huit colonnes affichent des objets identifiabiles (sac, pull/manteau, robe, bottine, t-shirt, robe). Entre les epochs 13 et 50, le raffinement se concentre sur les bords (silhouette plus nette du sac, contraste

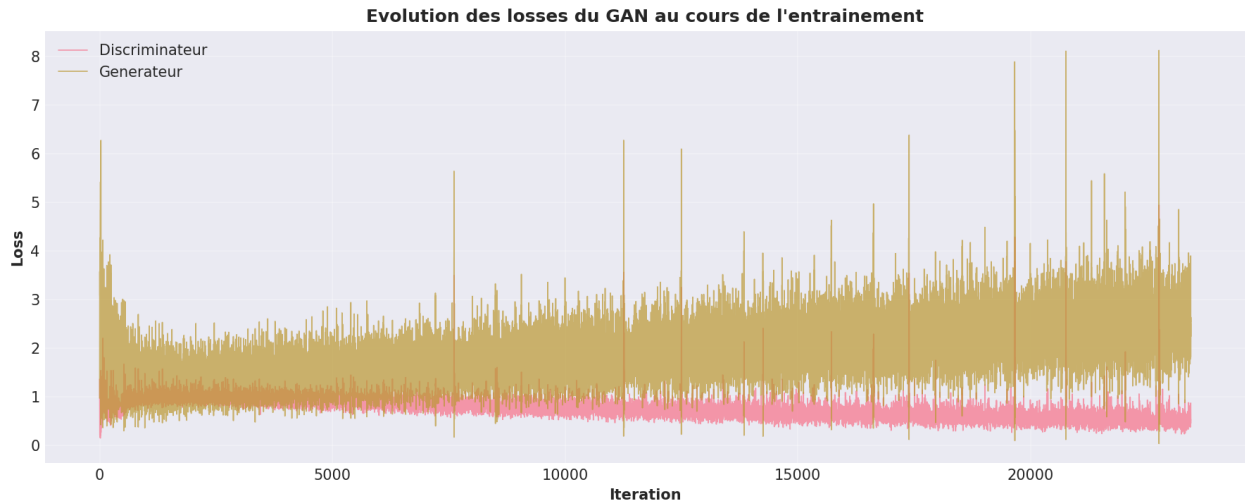


FIGURE 2 – Evolution des pertes D et G sur l’ensemble de l’entraînement. La D_loss reste stable autour de 0,5–1,0, la G_loss derive de 2 vers 3–4 avec des pics ponctuels au-delà de 6.

accru, structure du col du pull mieux dessinée). Le plafond qualitatif est atteint relativement tôt, conforme à ce qu’on attend d’un DCGAN sur ce dataset.

Cas pathologique notable. La cinquième colonne de la figure 3 montre un cas typique d’*echec local du generateur*. Le point latent en question donne lieu, au fil des epochs, à une image de plus en plus structurée par un motif damier de haute fréquence sans sémantique reconnaissable. Le générateur a appris à placer cette texture pour ce vecteur latent précis, mais elle ne correspond à aucune classe de Fashion-MNIST. C’est un sous-cas typique de *mode failure* ponctuel.

La grille de 100 images (figure 4) montre une diversité réelle : on identifie des sacs, des pantalons, des bottines (de hauteur variable), des sneakers, des chaussures à talon, des hauts et des robes. **Pas de mode collapse global**, mais des signaux locaux : (i) sur-représentation visuelle des classes structurellement simples (silhouette de pantalon, sac), (ii) plusieurs cases qui contiennent des silhouettes hybrides T-shirt/Chemise/Pull non identifiables, (iii) des chaussures avec des bouts mal définis, et (iv) quelques cases très claires ressemblant à des halos sans structure précise.

3.4 Réponses aux questions d’analyse

Realisme au cours de l’entraînement. Oui, et la transition est très rapide : entre les epochs 1 et 7, on passe de blobs à des objets identifiables. Les 40 epochs suivantes apportent un raffinement marginal des bords et du contraste.

Diversité. Correcte mais inférieure à celle du dataset réel. Sur 100 images, je compte plusieurs sacs (au moins 8), des pantalons (au moins 10), des chaussures variées (sneakers, bottines, talons), des hauts variés. Certaines classes (Sandale notamment) sont sous-représentées dans les sorties.

Mode collapse. Pas de collapse total. En revanche des signes locaux : répétitions de silhouettes voisines, et le cas pathologique de la colonne 5 de la figure 3 qui se “fige” sur un motif sans contenu sémantique.

Defauts les plus fréquents. Flou sur les bords (artefacts caractéristiques des `ConvTranspose2d` à `stride 2`), manches et chaussures asymétriques, semelles mal définies, hybridations entre classes proches (T-shirt/Chemise/Pull), et quelques images quasi-uniformes.

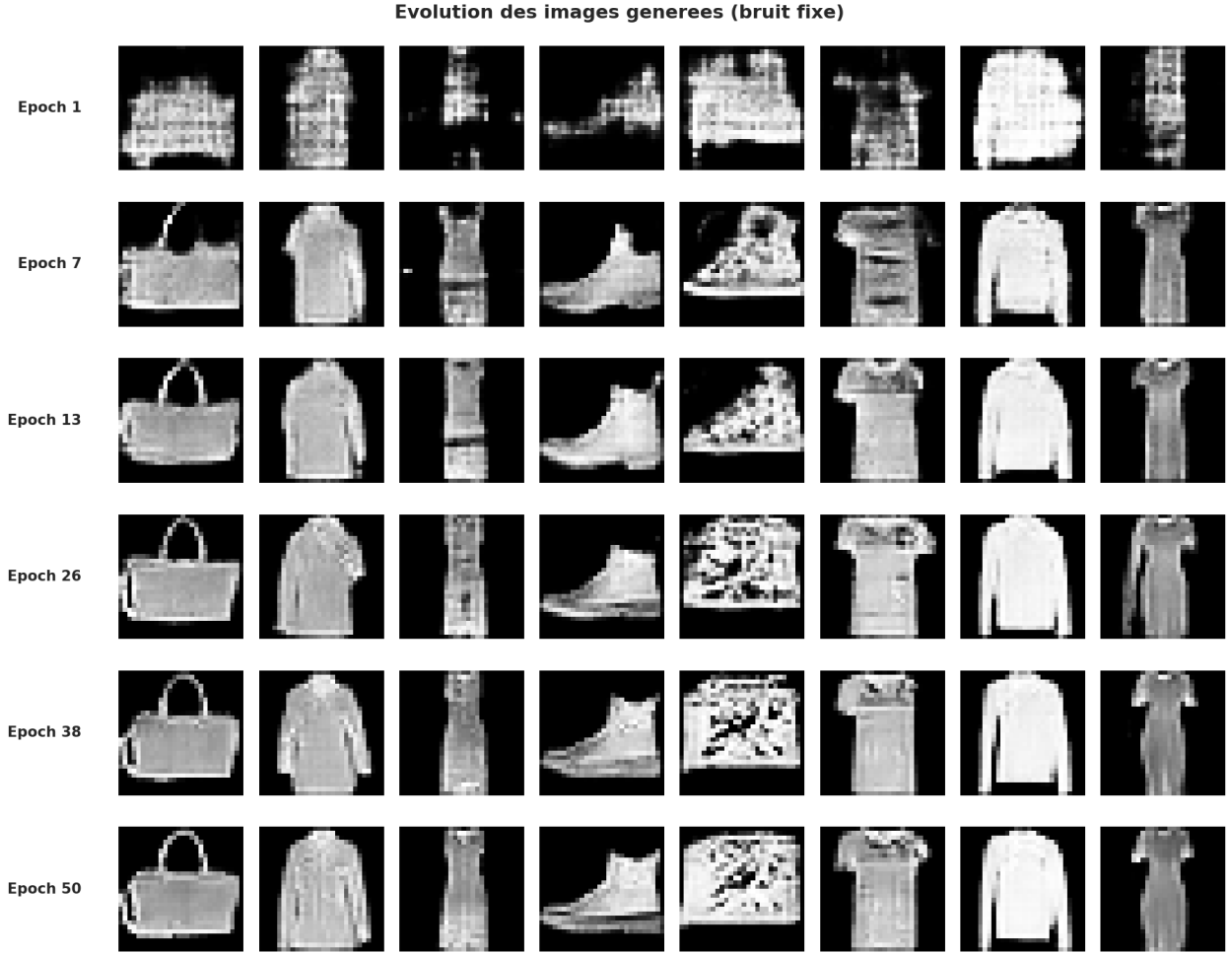


FIGURE 3 – Evolution des images generees a partir du *meme* bruit fixe sur 6 epochs reparties. Seul le generateur change d’une ligne a l’autre.

Interpretabilité des losses. Faible. Les losses servent surtout a detecter les pathologies (divergence, ecrasement de G , mode collapse). L’évaluation réelle de la qualité passe par l’inspection visuelle ou des metriques spécifiques (FID, IS) non implementees dans ce mini-projet.

4 Bilan et choix techniques

Trois choix sont a souligner. (1) Avoir adopte la baseline DCGAN plutot qu’une architecture personnalisee m’a permis d’obtenir un entraînement stable des le premier essai, et de me concentrer sur l’analyse plutot que sur le debogage. (2) L’utilisation du *bruit fixe* pour les snapshots est ce qui donne sa valeur diagnostique a la figure 3 : sans cela, on melange l’effet de l’apprentissage et l’effet du sampling, et on ne peut pas distinguer la convergence d’un cas d’echec local. (3) `BCEWithLogitsLoss` plutot que `Sigmoid + BCELoss` : c’est un detail d’implementation mais il evite des explosions numeriques quand le discriminateur produit des logits extremes, ce qui arrive frequemment a la fin de l’entraînement (voir les pics de la figure 2).

Le rapport general developpe plus en detail le positionnement de cette partie GAN dans le cadre comparatif global du projet (vs. diffusion).



FIGURE 4 – Grille finale : 100 images independantes generees apres entrainement.